

FarmaSys — Gestión Pro

Plan de Implementación del Backend

Spring Boot 3 · PostgreSQL · API REST

Versión 1.0 — Mayo 2026

Guía contract-first para construir el servidor
sin romper el frontend Angular ya existente.

Tabla de contenidos

1. Resumen y Principio Rector

Este documento define cómo construir el backend de FarmaSys de manera que encaje exactamente con el frontend Angular que ya existe. El frontend está aproximadamente al 75% y consume una API que aún no existe; el objetivo de este plan es que el servidor se construya correctamente a la primera, sin retrabajos por incompatibilidades.

1.1 El principio que evita la mayoría de problemas: Contract-First

El frontend ya define el contrato. Las interfaces TypeScript en `src/app/core/models` (Usuario, Producto, Cliente, Venta, Sucursal) NO son sugerencias: son la especificación exacta de lo que el backend debe devolver. Cada campo JSON que produzca el servidor debe tener el mismo nombre y el mismo tipo que la interfaz TypeScript correspondiente.

Si el backend devuelve "price" donde el frontend espera "precioVenta", o un número donde espera un string, la pantalla se rompe en silencio. Por eso la regla número uno es: el backend se adapta al contrato del frontend, nunca al revés.

1.2 Reglas de oro

- **Nombres en español:** las entidades y DTOs del backend usan los mismos nombres de campo que el frontend (nombres, apellidos, precioVenta, stockTotal). No se traduce a inglés.
- **IDs como UUID:** el frontend tipa todos los id como string. Se usa UUID en la base de datos, que serializa a string de forma natural.
- **Fechas en ISO 8601:** todos los campos de fecha y hora se serializan como string ISO (ej. 2026-05-20T14:30:00Z).
- **Dinero como número decimal:** precios y totales viajan como número JSON, no como string. En Java se usa BigDecimal y se configura Jackson para serializar a número.
- **Errores con forma fija:** toda respuesta de error debe ser { "mensaje": "..." }. El interceptor del frontend lee `err.error.mensaje`.
- **Prefijo de rutas:** todos los endpoints viven bajo `/api/v1`. Está fijado en `environment.ts` del frontend.

2. Stack Tecnológico y Dependencias

Versiones recomendadas, alineadas con el plan maestro y con lo que el frontend ya espera (environment.ts apunta a <http://localhost:8080/api/v1>).

Componente	Versión / Elección	Por qué
Java	JDK 21 (LTS)	Soporte de largo plazo; compatible con Spring Boot 3.
Spring Boot	3.3.x	Framework base; última rama estable.
Gestor de build	Maven	Estándar, documentación abundante.
Base de datos	PostgreSQL 16	Definido en el plan maestro.
ORM	Spring Data JPA + Hibernate	Productividad alta sobre PostgreSQL.
Migraciones	Flyway	Versionado del esquema en el repositorio.
Seguridad	Spring Security + JWT	Stateless, encaja con la SPA.
Librería JWT	jjwt (io.jsonwebtoken)	Generación y validación de tokens.
Validación	Bean Validation (Jakarta)	Anotaciones @NotNull, @Email, etc.
Documentación	springdoc-openapi	Swagger UI generado del código.
Mapeo DTO	MapStruct (opcional)	Convierte Entidad a DTO sin código repetitivo.
Contenedores	Docker + Docker Compose	PostgreSQL local y despliegue reproducible.
Pruebas	JUnit 5 + Spring Test + Testcontainers	Pruebas de lógica e integración con BD real.

2.1 Dependencias Maven (starters principales)

```

spring-boot-starter-web      // API REST
spring-boot-starter-data-jpa // JPA + Hibernate
spring-boot-starter-security // Spring Security
spring-boot-starter-validation // Bean Validation
postgresql                  // driver JDBC
flyway-core                  // migraciones
io.jsonwebtoken:jjwt-api/impl // JWT
springdoc-openapi-starter-webmvc-ui // Swagger

```

3. Estructura del Proyecto

Monolito modular: un solo proyecto desplegable, organizado en paquetes por dominio. Cada dominio agrupa su controller, service, repository, entidad y DTOs. Las dependencias entre dominios pasan por interfaces de servicio, nunca por acceso directo a entidades ajenas.

```
com.farmasys
├── config/      seguridad, CORS, OpenAPI, Jackson
├── security/    filtro JWT, UserDetails, TokenService
├── shared/      excepciones, respuesta de error, DTOs base, auditoría
├── auth/        login, refresh, registro de usuarios
├── empresa/     datos de la empresa (singleton) y sucursales
├── usuario/     usuarios, roles, permisos
├── catalogo/    productos, categorías, laboratorios, proveedores
├── inventario/  stock, lotes, movimientos, kardex, alertas
├── cliente/     clientes y recetas
├── venta/       POS, ventas, comprobantes, caja
└── reporte/     reportes operativos y financieros
```

Cada paquete de dominio sigue la misma estructura interna:

```
com.farmasys.catalogo
├── ProductoController.java  expone /api/v1/productos
├── ProductoService.java    lógica de negocio
├── ProductoRepository.java  acceso a datos (Spring Data)
├── Producto.java           entidad JPA
└── dto/ ProductoDTO, CrearProductoRequest, ...
```

3.1 Capas y su responsabilidad

- **Controller:** recibe la petición HTTP, valida el cuerpo, delega al service, devuelve un DTO. Sin lógica de negocio.
- **Service:** toda la lógica de negocio (cálculo de IGV, validación de stock, FEFO, generación de comprobantes). Es transaccional.
- **Repository:** solo acceso a datos. Interfaces de Spring Data JPA.
- **Entidad:** mapea una tabla. Nunca se expone directamente en la API.
- **DTO:** lo que viaja por la API. Su forma debe coincidir con la interfaz TypeScript del frontend.

4. Contrato de API — Convenciones Obligatorias

Esta es la sección más importante del documento. Respetar estas convenciones es lo que evita que el frontend se rompa.

4.1 Formato de respuesta de error

El interceptor del frontend (jwt.interceptor.ts) lee el mensaje de error desde `err.error.mensaje`. Por lo tanto, TODA respuesta de error del backend debe tener esta forma exacta:

```
{
  "mensaje": "El producto ya existe en el catálogo",
  "codigo": "RECURSO_DUPLICADO",
  "timestamp": "2026-05-20T14:30:00Z",
  "ruta": "/api/v1/productos"
}
```

Esto se logra con un `@RestControllerAdvice` global que captura todas las excepciones y las transforma a este formato. Sin esto, Spring devuelve su formato por defecto y el frontend mostrará mensajes genéricos o vacíos.

4.2 Códigos de estado HTTP

El interceptor ya maneja cada código. El backend debe usarlos correctamente:

Código	Cuándo usarlo	El frontend hace...
200 / 201	Operación exitosa / recurso creado	Procesa la respuesta normal.
400	Datos inválidos (validación falló)	Muestra el mensaje de error.
401	Token ausente, inválido o expirado	Cierra sesión y va a <code>/login</code> .
403	Autenticado pero sin permiso	Muestra "No tiene permisos".
404	Recurso no encontrado	Muestra "El recurso no existe".
409	Conflicto (duplicado, stock insuficiente)	Muestra <code>err.error.mensaje</code> .
500	Error no controlado del servidor	Muestra "Error interno".

4.3 Paginación

El frontend (FiltroVentas) envía `page` y `size`. Las respuestas de listas grandes deben paginarse. Se recomienda devolver el `Page` de Spring, que el frontend consumirá así:

```
{
  "content": [ ... ], // array de elementos
  "totalElements": 1240,
  "totalPages": 25,
  "number": 0,        // página actual (base 0)
  "size": 50
}
```

4.4 CORS

Error clásico que cuesta horas detectar: si no se configura CORS, el navegador bloquea todas las peticiones del frontend y los errores no son evidentes. El backend debe permitir explícitamente el origen del frontend:

```
// config/CorsConfig.java
allowedOrigins: http://localhost:4200 (desarrollo)
                 https://app.farmasys.pe (producción)
allowedMethods: GET, POST, PUT, PATCH, DELETE, OPTIONS
allowedHeaders: Authorization, Content-Type
```

4.5 Serialización JSON

- **Nombres de campo:** idénticos a las interfaces TypeScript. Si una entidad Java necesita otro nombre interno, se usa `@JsonProperty`.
- **Fechas:** configurar Jackson para escribir ISO 8601 (`WRITE_DATES_AS_TIMESTAMPS = false`).
- **Decimales:** `BigDecimal` serializa como número. No activar escritura como string.
- **Campos nulos:** se permiten; el frontend tipa muchos campos como opcionales (apellidos?, imageUrl?).

5. Inventario de Endpoints

Lista de endpoints derivada de los servicios Angular que ya existen. Cada uno indica qué servicio del frontend lo consume. Construir exactamente estos — ni más ni menos para la versión 1.

5.1 Autenticación

Método y ruta	Cuerpo / Respuesta	Consumido por
POST /auth/login	LoginRequest → LoginResponse	AuthService.login()
POST /auth/refresh	refreshToken → LoginResponse	AuthService (pendiente)
POST /auth/logout	invalida el refreshToken	AuthService.logout()

5.2 Catálogo e Inventario

Método y ruta	Cuerpo / Respuesta	Consumido por
GET /productos?activo=true	→ Producto[]	ProductoService.cargar()
GET /productos/{id}	→ Producto	ProductoService.porId()
POST /productos	CrearProductoRequest → Producto	InventarioService
PUT /productos/{id}	actualizar → Producto	InventarioService
GET /inventario/lotos	→ Lote[]	InventarioService
POST /inventario/ingreso	registra mercadería	InventarioService
POST /inventario/ajuste	ajuste con motivo	InventarioService

5.3 Ventas y Clientes

Método y ruta	Cuerpo / Respuesta	Consumido por
POST /ventas	VentaRequest → VentaResponse	VentaService.registrar()
GET /ventas	FiltroVentas → Page<Venta>	VentaService.historial()
GET /ventas/{id}	→ Venta	VentaService
POST /ventas/{id}/anular	motivo → Venta	VentaService.anular()
GET /clientes?q=...	→ Cliente[]	ClienteService.buscar()
GET /clientes/documento/{nro}	→ Cliente	ClienteService
POST /clientes	crear → Cliente	ClienteService.registrar()

5.4 Sucursales, Usuarios y Reportes

Método y ruta	Cuerpo / Respuesta	Consumido por
GET /sucursales	→ Sucursal[]	SucursalService
GET /sucursales/{id}	→ Sucursal (con KPI)	SucursalService
GET /usuarios	→ Usuario[]	UsuariosComponent
POST /usuarios	crear → Usuario	UsuariosComponent
GET /reportes/ventas	rango → resumen	FinanzasComponent
GET /reportes/dashboard	→ KPIs	DashboardComponent

6. Esquema de Base de Datos

Tablas centrales. Cada tabla incluye implícitamente: id (UUID, PK), creado_en, actualizado_en, creado_por, actualizado_por. Las tablas operativas llevan sucursal_id para el filtrado multi-sucursal.

Tabla	Columnas clave	Notas
empresa	razon_social, ruc, nombre_comercial	Una sola fila (singleton).
sucursal	nombre, direccion, distrito, telefono, estado, apertura	N filas; FK a empresa.
usuario	email (único), password_hash, nombres, apellidos, activo, sucursal_id	password_hash con BCrypt.
rol	nombre, descripcion	SUPER_ADMIN, ADMIN, etc.
usuario_rol	usuario_id, rol_id	Relación N:M.
producto	codigo (único), nombre, principio_activo, categoria, precio_venta, requiere_receta, controlado, activo	codigo único por empresa.
lote	numero_lote, fecha_vencimiento, costo_unitario, stock_actual, producto_id, sucursal_id	Stock real vive aquí.
movimiento_stock	tipo, cantidad, fecha, motivo, lote_id	Entradas, salidas, ajustes.
cliente	tipo_documento, numero_documento, nombres, apellidos, razon_social, puntos	numero_documento único.
venta	numero_comprobante, tipo_comprobante, fecha_hora, estado, subtotal, igv, total, metodo_pago, cliente_id, sucursal_id, cajero_id	Cabecera de la venta.
venta_item	producto_id, lote_id, cantidad, precio_unitario, descuento, subtotal, venta_id	Detalle de la venta.
caja	fecha_apertura, monto_inicial, fecha_cierre, monto_final, sucursal_id, usuario_id	Apertura y cierre diario.

6.1 Reglas del esquema

- **Claves foráneas explícitas:** toda relación se declara con FK y su índice correspondiente.
- **Índices desde el inicio:** sucursal_id, fecha_hora, producto_id, cliente_id, numero_documento. Sin estos, las consultas se degradan rápido.
- **Unicidad con alcance:** codigo de producto y numero_documento de cliente son únicos a nivel de empresa.
- **Stock por lote:** el stock no es una columna del producto; es la suma de los lotes. El producto.stockTotal del frontend es un valor calculado que el backend agrega.
- **Borrado lógico:** productos y usuarios se desactivan con un flag activo, no se eliminan físicamente.

7. Seguridad y Autenticación JWT

7.1 Flujo de autenticación

- El usuario envía email y password a POST /auth/login.
- El backend valida contra la tabla usuario (password con BCrypt).
- Si es correcto, genera un accessToken (corta duración) y un refreshToken (larga duración).
- El accessToken incluye en sus claims: id de usuario, email, roles y sucursal_id.
- El frontend guarda ambos y envía el accessToken en cada petición (header Authorization).
- Un filtro de Spring Security valida el token en cada request y arma el contexto de seguridad.

7.2 Duración de tokens

El frontend (environment.ts) tiene tokenExpiracionSeg = 3600. El accessToken debe durar exactamente eso: 1 hora. El refreshToken debe durar más (por ejemplo 7 días) para permitir renovar la sesión sin volver a pedir contraseña.

- **accessToken:** 1 hora. Corto a propósito: si se roba, expira pronto.
- **refreshToken:** 7 días. Permite que POST /auth/refresh entregue un accessToken nuevo sin login.

7.3 Endpoints públicos vs privados

El interceptor del frontend ya trata /auth/login y /auth/refresh como públicos. El backend debe configurar Spring Security igual:

- **Públicos (sin token):** /auth/login, /auth/refresh, Swagger UI, health check.
- **Privados (requieren token):** todo lo demás bajo /api/v1.

7.4 Autorización por rol

El frontend ya tiene rolGuard por ruta. El backend debe replicar la misma autorización en sus endpoints con @PreAuthorize — la seguridad del frontend es solo cosmética, la verdadera barrera está en el servidor.

```
@PreAuthorize("hasAnyRole('ADMIN','VENDEDOR')")
POST /api/v1/ventas // solo cajeros y administradores
```

```
@PreAuthorize("hasRole('SUPER_ADMIN')")
GET /api/v1/sucursales // solo el super administrador
```

SUPER_ADMIN debe poder pasar todos los chequeos de rol, igual que en el frontend. Esto se logra con una jerarquía de roles de Spring Security donde SUPER_ADMIN hereda de todos.

7.5 Aislamiento multi-sucursal

Riesgo serio: un ADMIN de la sucursal A no debe poder ver datos de la sucursal B. Regla obligatoria:

- **Filtro por sucursal:** todo servicio que consulte datos operativos filtra por la sucursal_id del usuario autenticado (tomada del token), salvo que el usuario sea SUPER_ADMIN.

- **Nunca confiar en el frontend:** aunque VentaRequest trae sucursalId, el backend debe verificar que coincide con la del token. Un usuario no puede registrar ventas en una sucursal ajena.

8. Migraciones con Flyway

Toda la estructura de la base de datos vive en scripts SQL versionados dentro del repositorio (src/main/resources/db/migration). Nunca se modifica la base de datos a mano. Flyway aplica los scripts en orden al arrancar la aplicación.

Script	Contenido
V1__esquema_inicial.sql	Tablas empresa, sucursal, usuario, rol, usuario_rol.
V2__catalogo.sql	Tablas producto, categoria, laboratorio, proveedor.
V3__inventario.sql	Tablas lote, movimiento_stock.
V4__cliente.sql	Tablas cliente, receta.
V5__venta.sql	Tablas venta, venta_item, caja, movimiento_caja.
V6__indices.sql	Índices de rendimiento sobre las FK y columnas de filtro.
V7__datos_iniciales.sql	Empresa, sucursales y usuario SUPER_ADMIN semilla.

- **Regla de oro:** una vez que un script V_ se aplicó en cualquier entorno, NUNCA se modifica. Los cambios posteriores van en un script nuevo (V8, V9, ...).

9. Manejo de Errores y Validación

9.1 Manejador global de excepciones

Un único `@RestControllerAdvice` traduce todas las excepciones al formato { mensaje, código, timestamp, ruta }. Casos a cubrir:

- **Validación fallida (400):** cuerpo de petición que no cumple las anotaciones Bean Validation.
- **Recurso no encontrado (404):** excepción propia `RecursoNoEncontradoException`.
- **Conflicto de negocio (409):** duplicados, stock insuficiente — excepción `ReglaNegocioException`.
- **Acceso denegado (403):** `AccessDeniedException` de Spring Security.
- **No controlado (500):** cualquier otra; se registra en log con detalle, se devuelve mensaje genérico.

9.2 Validación de entrada

Cada DTO de petición lleva anotaciones de Bean Validation. El controller las activa con `@Valid`. Ejemplos:

```
public class LoginRequest {  
    @NotBlank @Email String email;  
    @NotBlank String password;  
}
```

9.3 Validaciones de negocio que el backend debe imponer

El frontend ya valida algunas cosas, pero el backend NO debe confiar en eso. Debe revalidar siempre:

- **Stock suficiente:** no se puede vender más unidades de las que hay en lotes disponibles.
- **Factura requiere RUC:** una venta tipo FACTURA exige un cliente con documento RUC.
- **Producto controlado:** venta de controlados exige datos de receta y cliente identificado.
- **Coherencia de totales:** el backend recalcula subtotal, IGV y total; no confía en los números que envía el frontend.
- **FEFO:** al descontar stock, se consume primero el lote con vencimiento más próximo.

10. Roadmap del Backend por Fases

Orden de construcción. Cada fase deja el backend en un estado verificable y permite conectar una parte del frontend.

10.1 Fase B0 — Cimientos (3-4 días)

- Crear proyecto Spring Boot 3 con la estructura modular.
- Docker Compose con PostgreSQL 16.
- Configurar Flyway, Jackson (fechas ISO), CORS y OpenAPI.
- Manejador global de excepciones con el formato { mensaje, ... }.
- Migración V1 (empresa, sucursal, usuario, rol).

10.2 Fase B1 — Autenticación (1 semana)

- Spring Security + filtro JWT + TokenService (jjwt).
- POST /auth/login, /auth/refresh, /auth/logout.
- Jerarquía de roles con SUPER_ADMIN heredando todo.
- Datos semilla: empresa, 5 sucursales, usuarios de cada rol.
- Conectar AuthService del frontend (activar el bloque REAL).

10.3 Fase B2 — Catálogo e Inventario (1-2 semanas)

- Migraciones V2 y V3; entidades Producto, Lote, MovimientoStock.
- GET /productos, /productos/{id}; CRUD de productos.
- Endpoints de ingreso de mercadería y ajuste de stock.
- Cálculo de stockTotal agregando lotes.
- Conectar ProductoService e InventarioService del frontend.

10.4 Fase B3 — Ventas y Caja (1-2 semanas)

- Migración V5; entidades Venta, Ventaltem, Caja.
- POST /ventas con validación de stock, IGV, FEFO y descuento de lotes.
- GET /ventas con filtros y paginación; anulación de ventas.
- Apertura y cierre de caja.
- Conectar VentaService del frontend.

10.5 Fase B4 — Clientes y Reportes (1 semana)

- Migración V4; entidad Cliente y Receta.
- CRUD de clientes; búsqueda por documento.
- Endpoints de reportes para Dashboard y Finanzas.
- Conectar ClienteService, DashboardComponent y FinanzasComponent.

10.6 Fase B5 — Hardening y despliegue (1 semana)

- Pruebas de la lógica crítica (IGV, FEFO, stock) con JUnit.
- Pruebas de integración con Testcontainers.
- Dockerizar el backend; Docker Compose completo (API + BD).

- Despliegue a un servidor de pruebas; revisión de rendimiento.

11. Riesgos y Decisiones Críticas

Errores comunes que generan problemas costosos si no se deciden antes de escribir código.

Riesgo	Cómo se evita
Nombres de campo distintos al frontend	Nombrar entidades y DTOs en español igual que las interfaces TypeScript. Revisar contra los .model.ts.
Olvidar CORS	Configurar CorsConfig en la Fase B0, antes de cualquier endpoint.
Formato de error incompatible	Implementar el @RestControllerAdvice en la Fase B0, no después.
Exponer entidades JPA en la API	Siempre devolver DTOs. La entidad nunca sale del service.
Fuga de datos entre sucursales	Filtrar por sucursal_id del token en cada consulta operativa.
Confiar en los totales del frontend	El backend recalcula subtotal, IGV y total siempre.
Modificar una migración ya aplicada	Toda corrección va en un script Flyway nuevo.
Stock negativo por ventas concurrentes	Bloqueo optimista o pesimista al descontar lotes dentro de la transacción.
Contraseñas en texto plano	BCrypt siempre. Nunca guardar ni loguear contraseñas.

11.1 Decisiones a confirmar antes de empezar

- Confirmar versión exacta de Spring Boot 3.3.x disponible al momento de iniciar.
- Definir dónde correrá PostgreSQL en producción (mismo servidor o gestionado).
- Acordar la política de expiración: 1 h de accessToken es lo recomendado.
- Definir si los reportes pesados se calculan en vivo o se cachean.

12. Checklist Pre-Código

Antes de escribir el primer controller, verificar que estas decisiones de cimientos estén tomadas e implementadas. Saltarse esto es la causa número uno de retrabajo.

- Proyecto Spring Boot 3 creado con la estructura modular por dominios.
- Docker Compose con PostgreSQL 16 levantando correctamente.
- Flyway configurado y la migración V1 aplicándose al arrancar.
- Jackson configurado: fechas en ISO 8601, decimales como número.
- CORS permitiendo `http://localhost:4200`.
- `@RestControllerAdvice` global devolviendo { mensaje, codigo, timestamp, ruta }.
- Spring Security con `/auth/**` público y el resto protegido.
- Jerarquía de roles con `SUPER_ADMIN` heredando de todos.
- `springdoc-openapi` accesible en `/swagger-ui` para probar endpoints.
- Las entidades y DTOs revisados campo por campo contra los `.model.ts` del frontend.

Cuando los diez puntos estén marcados, el backend tiene cimientos correctos y se puede construir cada dominio sin sorpresas.

Aprobado por: _____ Fecha: _____